

Dream Games

Software Engineering Study

In this study, you are expected to create a simple, level-based mobile game. Implementation should be done in **Unity** and **C#**. The game must be playable on Unity Editor.

Flow

- When the user launches the game, s/he will see a **LevelButton** in the **MainScene**. LevelButton displays the user's current level. When all levels are finished, the LevelButton should display finished text.
- The background of the MainScene should be an area image.
- After tapping the LevelButton, **LevelScene** should be loaded according to the current level. (You can find gameplay details in the Gameplay section)
- When a user wins the level
 - Celebration particles and animation should be shown to the user
 - MainScene should be loaded
- When a user loses the level
 - A fail popup should be shown to the user, which has options to return to MainScene with a close button and replay the level with a try again button.
- The user's last played level number should be persisted locally. When Unity is restarted, the LevelButton should display the persisted level number.
- A Unity editor menu item should be implemented to set the last played level number.



Level 1



All levels are finished

Gameplay

The goal is to clear all obstacles by blasting cubes near them or using Rockets.

Mechanics

- There is a rectangular grid where each cell can have one item. The grid can have a width and height of six to ten cells.
- There are four types (colors) of **Cubes**. At least 2 adjacent cubes with the same color can be blasted by tapping any cube in the group.
- Adjacent cubes should be direct neighbors, either horizontally or vertically.
- All falling objects can fall down to empty cells only in the vertical direction. New random cubes should fall from atop the grid. Fall mechanics should be implemented by coding, not Physics or Unity animations. Cubes cannot fall through other cubes, rockets, or obstacles.
- A move is spent after a valid tap operation.
- The level will be won if all obstacles are cleared within a given move count.

Rocket

- When the blasted cube group count is bigger than or equal to 4, cubes need to animate to the clicked cell and create a **Rocket**.
- The rocket's direction should be horizontal or vertical randomly.
- The rocket can fall down to empty cells.
- The rocket gets created at the location of the tapped cube.
- The rocket explodes when tapped or gets damaged by another rocket.
- The rocket gets divided into 2 moving parts in opposite directions when it explodes. The parts will damage cells as they go over them.
- Cube groups should display a rocket logo on them if they are eligible to create one.
- A Rocket will be combined with other adjacent Rockets to create a **Combo** when tapped. Rocket-Rocket combo will create 3x3 exploding rockets in both directions.

Obstacle

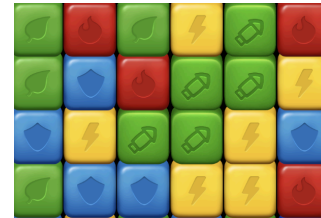
- Obstacles are goals of levels. Users should clear all obstacles to win the level within a given move count. There are three types of obstacles.
- **The box** takes damage when a blast occurs in adjacent cells or a Rocket goes over them. A box can be cleared with one damage and does not fall down to empty cells.
- **Stone** takes damage only when a Rocket goes over them. Stone can be cleared with one damage. Stone does not fall down to empty cells.
- **The vase** takes damage when a blast occurs in adjacent cells, or a Rocket goes over them. The vase can be cleared with two damages. The vase takes no more than one damage from a single blasted cube group. The vase takes one damage from each separate Rocket. Vases can fall down to empty cells.



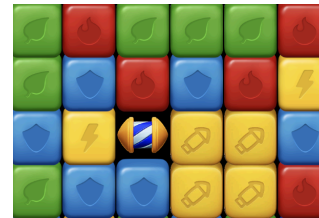
Board with all items



Rocket combo



Rocket Hint



Rocket created on the clicked cell

Levels

All 10 levels should be included in the project to make them available to users for the initial launch. The level file structure is as follows;

- **level_number**: number of the level
- **grid_width**: width of the grid
- **grid_height**: height of the grid
- **move_count**: given move count for the level
- **grid**: a list of items (**r**: Red, **g**: Green, **b**: Blue, **y**: Yellow, **rand**: One of random colors(r, g, b, y), **vro**: Vertical Rocket, **hro**: Horizontal Rocket, **bo**: Box, **s**: Stone, **v**: Vase) starts from the **bottom left** of the grid and continues horizontally, ends at the **top right** of the grid.

Notes

- **Unity 6000.0.32f1** and **C#** should be used in the implementation with Unity's built-in renderer.
- Your code should follow **Object Oriented Programming** (OOP) principles such as encapsulation, inheritance, and polymorphism.
- The structure should be well-designed to make new additions and changes easily.
- You can use third-party tween libraries for animations.
- You shouldn't use third-party dependency injection libraries like Zenject.
- The game should support portrait (9:16) orientation.
- Try to make nice animations as best as you can.
- You can analyze Royal Match, Royal Kingdom and also similar blast games for animations.

Submission

You need to submit a Unity project from the editor that is playable. You need to share a private GitHub repository with software.engineering.study@dreamgames.com.